

Eliminating Conflict Misses Using Prime Number-Based Cache Indexing

Mazen Kharbutli, *Student Member, IEEE*, Yan Solihin, *Member, IEEE*, and Jaejin Lee, *Member, IEEE*

Abstract—Using alternative cache indexing/hashing functions is a popular technique to reduce conflict misses by achieving a more uniform cache access distribution across the sets in the cache. Although various alternative hashing functions have been demonstrated to eliminate the worst-case conflict behavior, no study has really analyzed the pathological behavior of such hashing functions that often results in performance slowdown. In this paper, we present an in-depth analysis of the pathological behavior of cache hashing functions. Based on the analysis, we propose two new hashing functions, *prime modulo* and *odd-multiplier displacement*, that are resistant to pathological behavior and yet are able to eliminate the worst-case conflict behavior in the L2 cache. We show that these two schemes can be implemented in fast hardware using a set of narrow addition operations, with negligible fragmentation in the L2 cache. We evaluate the schemes on 23 memory intensive applications. For applications that have nonuniform cache accesses, both prime modulo and odd-multiplier displacement hashing achieve an average speedup of 1.27 compared to traditional hashing, without slowing down any of the 23 benchmarks. We also evaluate using odd-multiplier displacement function with multiple multipliers in conjunction with a skewed associative L2 cache. The skewed associative cache achieves a better average speedup at the cost of some pathological behavior that slows down four applications by up to 7 percent.

Index Terms—Cache hashing, cache indexing, prime modulo, odd-multiplier displacement, conflict misses.

1 INTRODUCTION

DESPITE the relatively large size and high associativity of the L2 cache, conflict misses are a significant performance bottleneck in many applications. Alternative cache indexing/hashing functions are used to reduce such conflicts by achieving a more uniform access distribution across the L1 cache sets [18], [4], [23], [22], L2 cache sets [19], or the main memory banks [15], [11], [16], [17], [20], [8], [7], [26], [12]. Although various alternative hashing functions have been demonstrated to eliminate the worst-case conflict behavior, few studies, if any, have analyzed the pathological behavior of such hashing functions that often result in performance degradation.

This paper presents an in-depth analysis of the pathological behavior of hashing functions and proposes two hashing functions, *prime modulo* and *odd-multiplier displacement* that are resistant to the pathological behavior and yet able to eliminate the worst-case conflict misses from the L2 cache. The number of cache sets in the prime modulo hashing is a prime number, while the odd-multiplier displacement hashing adds an offset, equal to an odd or prime number multiplied by the tag bits, to the index bits of an address to obtain a new cache index. The prime modulo hashing has been used in software hash tables [1] and in the Burroughs Scientific Processor [11]. A fast implementation of prime modulo hashing has only been proposed for

Mersenne prime numbers [25]. Since Mersenne prime numbers are sparse (i.e., for most n , $2^n - 1$ are not prime), using Mersenne prime numbers significantly restricts the number of cache sets that can be implemented.

We present an implementation that solves the three main drawbacks of the prime modulo hashing when it is directly applied to cache indexing. First, we present a fast hardware mechanism that uses a set of narrow add operations in place of a true integer division for any number of cache sets. Second, by applying the prime modulo to the L2 cache, the fragmentation that results from not fully utilizing a power of two number of cache sets becomes negligible. Finally, we show an implementation where the latency of the prime modulo computation can be hidden by performing it in parallel with L1 accesses and by caching the partial computation in the TLB. We show that the prime modulo hashing has properties that make it resistant to the pathological behavior that plagues other alternative hashing functions, while, at the same time, enabling it to eliminate worst-case conflict misses.

Although the odd-multiplier displacement hashing lacks the theoretical superiority of the prime modulo hashing, it can perform just as well in practice when the multiplier is carefully selected. In addition, the odd-multiplier displacement hashing can easily be used in conjunction with a skewed associative cache that uses multiple hashing functions to further distribute the cache set accesses. In this case, a unique multiplier is used for each cache bank.

We use 23 memory intensive applications from various sources and categorize them into two classes: one with nonuniform cache set accesses and the other with uniform cache set accesses. We found that, on the applications with nonuniform accesses, the prime modulo hashing achieves an average speedup of 1.27. It does not slow down any of the 23 applications except in one case by only 2 percent. The odd-multiplier displacement hashing achieves almost identical

- M. Kharbutli and Y. Solihin are with the Department of Electrical and Computer Engineering, North Carolina State University, Campus Box 7256, Raleigh, NC 27695-7256. E-mail: {mmkharbu, solihin}@ncsu.edu.
- J. Lee is with the School of Computer Science and Engineering, Room 505, Bldg. 301, Seoul National University, Seoul 151-742, Korea. E-mail: jlee@cse.snu.ac.kr.

Manuscript received 26 Mar. 2004; revised 19 Oct. 2004; accepted 15 Nov. 2004; published online 16 Mar. 2005.

For information on obtaining reprints of this article, please send e-mail to: tc@computer.org, and reference IEEECS Log Number TC-0107-0304.

performance to the prime modulo hashing, but without slowing down any of the 23 benchmarks. Both methods outperform an XOR-based indexing function, which obtains an average speedup of 1.21 on applications with nonuniform cache set accesses.

Using multiple hashing functions, skewed associative caches sometimes are able to eliminate more misses than using a single hashing function. However, we found that the use of a skewed associative cache introduces some pathological behavior that slows down some applications. A skewed associative cache with the XOR-based hashing [18], [4], [19] obtains an average speedup of 1.31 for benchmarks with non-uniform accesses, but slows down four applications by up to 9 percent. However, using the odd-multiplier displacement hashing that we propose in conjunction with a skewed associative cache, the average speedup improves to 1.35 and the worst-case slowdown improves to 7 percent.

The rest of the paper is organized as follows: Section 2 discusses the metrics and ideal properties of a hashing function, which help in understanding the pathological behavior. Section 3 discusses the proposed hashing functions: prime modulo and odd-multiplier displacement and their implementations. Section 4 describes the evaluation environment, while Section 5 discusses the results obtained. Section 6 lists the related work. Finally, Section 7 concludes the paper.

2 PROPERTIES OF HASHING FUNCTIONS

In this section, we will describe two metrics that are helpful in understanding the pathological behavior of hashing functions (Section 2.1) and two properties that a hashing function must have in order to avoid the pathological behavior (Section 2.2).

2.1 Metrics

The first metric that estimates the degree of the pathological behavior is *balance*, which describes how evenly distributed the addresses are over the cache sets. The other is *concentration*, which measures how evenly the sets are used over small intervals of accesses.

Let n_{set} be the number of sets in the cache that are a power of two. This implies that $\log_2 n_{set}$ bits from the address are used to obtain a set index. Let a sequence $\langle a_1, a_2, \dots, a_m \rangle$ of block addresses a_i denote m cache accesses. Suppose $a_i \neq a_j$ for $i \neq j$, where $1 \leq i, j \leq m$, implying that each block address in the sequence is unique. Thus, the address sequence does not have any temporal reuse (we will return to this issue later). Let H be a hashing function that maps each block address a_i to set $H(a_i)$ in the cache. We denote $\log_2 n_{set}$ index bits of a_i with x_i and the first $\log_2 n_{set}$ bits of the tag of a_i with t_i , as shown in Fig. 1.

Balance describes how evenly distributed the addresses are over the sets in the cache. When good balance is not achieved, the hashing function would be ineffective and would cause conflict misses. To measure the balance, we use a formula suggested by Aho and Ullman [1]:

$$balance = \frac{\sum_{j=1}^{n_{set}} \frac{b_j \cdot (b_j + 1)}{2}}{\frac{m}{2 \times n_{set}} \cdot (m + 2 \times n_{set} - 1)}, \quad (1)$$

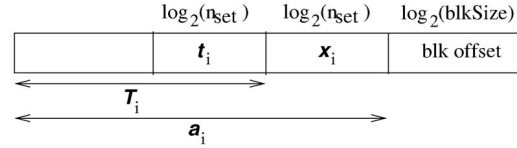


Fig. 1. Components of a block address a_i .

where b_j represents the total number of addresses that are mapped to set j . $\frac{b_j \cdot (b_j + 1)}{2}$ represents the weight of the set j , equivalent to $1 + 2 + \dots + b_j$. Thus, a set that has more addresses will have a larger weight. The numerator, $\sum_{j=1}^{n_{set}} \frac{b_j \cdot (b_j + 1)}{2}$, represents the sum of the weights of all sets. The denominator, $\frac{m}{2 \times n_{set}} \cdot (m + 2 \times n_{set} - 1)$, represents the sum of the weights of all sets, but assuming a perfectly random address distribution across all sets [1]. Thus, a *balance* value close to an ideal value of 1 represents better address distribution across the sets.

Concentration is a less straightforward measure and is intended to measure how evenly the sets are used over small intervals of accesses. It is possible to achieve the ideal balance for the entire address sequence $\langle a_1, a_2, \dots, a_m \rangle$, yet conflicts can occur if, on smaller intervals, the balance is not achieved.

To measure concentration, we calculate the distance d_i as the number of accesses to the cache that occur between two accesses to a particular set and calculate the standard deviation of these distances. More formally, d_i for an address a_i is the smallest positive integer such that $H(a_i) = H(a_{i+d_i})$. The concentration is equal to the standard deviation of d_i s. Noting that, in the ideal case and with the balance of 1, the average of d_i s is necessarily $n_{set}^{1/m}$, our formula is

$$concentration = \sqrt{\frac{\sum_{i=1}^m (d_i - n_{set})^2}{m}}. \quad (2)$$

Using standard deviation penalizes a hashing function not only for reaccessing a set after a small time period since its last access ($d_i < n_{set}$), but also for a large time period ($d_i > n_{set}$). The smaller the concentration is, the better the hashing function is. The concentration of an ideal hashing function is zero.

In general, alternative hashing functions have mostly targeted the ideal balance, but not the ideal concentration. Achieving good concentration is vital in avoiding pathological behavior for applications with high temporal locality. If a set receives a burst of accesses to many distinct addresses, then the set suffers from conflict misses temporarily. If one of the addresses has temporal reuse, it may have been replaced from the cache by the time it is reaccessed, creating conflict misses.

2.2 Ideal Properties

In this section, we describe the properties that should be satisfied by an ideal hashing function. Most applications, even some irregular applications, often have strided access patterns. Given the common occurrence of these patterns, a

1. There are m accesses spread ideally across n_{set} sets, so the total distance between accesses is $m \cdot n_{set}$. Hence, the average over the m accesses is n_{set} .

hashing function that does not achieve the ideal balance and concentration will cause a *pathological behavior*. A pathological behavior arises when the balance or concentration of an alternative hashing function is worse than those of the traditional hashing function, often leading to slowdown.

Let s be a stride amount in the address sequence $\langle a_1, a_2, \dots, a_m \rangle$, i.e., $a_{i+1} = a_i + s$, where $1 \leq i < m$.

Property 1 (Ideal balance). For the modulo-based hashing where $H(a_i) = a_i \bmod n_{set}$, the ideal balance is achieved if and only if $\gcd(s, n_{set}) = 1$, as shown in [15]. For other hashing functions, such as XOR-based, the ideal balance condition is harder to formulate because the hashing function has various cases where the ideal balance is not achieved (Section 3.3).

Property 2 (Sequence invariance). A hashing function is sequence invariant if and only if, for any a_i , $H(a_i) = H(a_{i+x})$ implies $H(a_{i+1}) = H(a_{i+x+1})$.

The ideal concentration is achieved when both the ideal balance and sequence invariance are satisfied.² Therefore, the ideal concentration is not achieved when the sequence invariance is not achieved. The sequence invariance says that once a set is reaccessed, the sequence of set accesses will precisely follow the previous sequence. Moreover, when the sequence invariance is satisfied, all the distances between two accesses to the same set are equal to a constant d , indicating the absence of a burst of accesses to a single set for the strided access pattern. Furthermore, when the ideal balance is satisfied for the modulo hashing, then the constant d is the average distance, or $d = x = n_{set}$.

It is possible that a hashing function satisfies the sequence invariance property in most, but not all, cases. Such a function can be said to have *partial sequence invariance*.

In Section 3.3, we will show that the prime modulo hashing function satisfies both properties except for a very small number of cases, whereas other hashing functions do not always achieve Property 1 and 2 simultaneously. Bad concentration is a major source of the pathological behavior for alternative hashing functions.

3 HASHING FUNCTIONS BASED ON PRIME NUMBERS

In this section, we describe the prime modulo and odd-multiplier displacement hashing functions that we propose (Sections 3.1 and 3.2, respectively). We compare them against other hashing functions in Section 3.3.

3.1 The Prime Modulo Hashing Function

Prime modulo hashing functions, like any other modulo functions, can be expressed as $H(a_i) = a_i \bmod n_{set}$. The difference between prime modulo hashing and traditional hashing is that n_{set} is a prime number instead of a power of two. The prime modulo hashing functions that have been used in software hash tables [1] and the BSP machine [11] have two major drawbacks. First, they are considered

TABLE 1
Prime Modulo Set Fragmentation

n_{set_phys}	n_{set}	Fragmentation (%)
256	251	1.95%
512	509	0.59%
1024	1021	0.29%
2048	2039	0.44%
4096	4093	0.07%
8192	8191	0.01%
16384	16381	0.02%

expensive to implement in hardware because performing a modulo operation with a prime number requires an integer division [11]. Second, since the number of sets in the physical memory (n_{set_phys}) is likely a power of two, there are $\Delta = n_{set_phys} - n_{set}$ sets that are wasted, causing fragmentation. For example, the fragmentation in the BSP is a nontrivial 6.3 percent. To minimize the fragmentation, n_{set} is generally chosen to be the largest prime number that is smaller than n_{set_phys} .

Since we target the L2 cache, however, this fragmentation becomes negligible. Table 1 shows that the percentage of the sets that are wasted in an L2 cache is small for commonly used numbers of sets in the L2 cache. The fragmentation falls below 1 percent when there are 512 physical sets or more. This is due to the fact that there is always a prime number that is very close to a power of two. In cases where the wasted sets are not fabricated, fragmentation is no longer an issue.

Utilizing number theory, we can compute the prime modulo function quickly without using an integer division. The foundation for the technique is taken from fast random number generators [14], [24]. Specifically, computing $2k \cdot x \bmod m$ where m is a Mersenne prime number (i.e., m is one less than a power of two) can be performed using add operations without a multiplication or division operation.

Since we are interested in a prime number n_{set} that is not necessarily Mersenne prime, we extend the existing method and propose two methods of performing the prime modulo operation fast without any multiplication and division. The first method, *iterative linear*, needs recursive steps of shift, add, and subtract&select operations. The second method, *polynomial*, needs only one step of add and a subtract&select operation.

Subtract&select method. Computing the value of $x \bmod n_{set}$ is trivial if x is small. Fig. 2 shows how this can be implemented in hardware. x , $x - n_{set}$, $x - 2n_{set}$, $x - 3n_{set}$, etc. are all fed as input into a selector which chooses the rightmost input that is not negative. To implement this method, the maximum value of x should be known.

Iterative linear method. First, a_i is represented as a linear function of $\Delta = n_{set_phys} - n_{set}$. To see how this can be done, let T_i and x_i represent parts of the bits in a_i , as depicted in Fig. 1. Since $a_i = n_{set_phys} \cdot T_i + x_i$, then³

3. Note that, while the expression $x = y \bmod z$ means that the modulo operation on y results in x , the expression $x = y (\bmod z)$ means that $x = y$ is true in the modulo space.

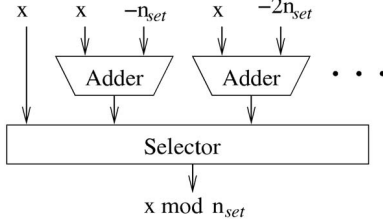


Fig. 2. Hardware implementation of the subtract&select method.

$$\begin{aligned}
 a_i &\equiv n_{set_phys} \cdot T_i + x_i \pmod{n_{set}} \\
 &\equiv (n_{set} + \Delta) \cdot T_i + x_i \pmod{n_{set}} \\
 &\equiv \Delta \cdot T_i + x_i \pmod{n_{set}} \\
 &\equiv a'_i \pmod{n_{set}}.
 \end{aligned} \tag{3}$$

Since a'_i is much smaller than a_i , $a'_i \pmod{n_{set}}$ may be computed using the subtract&select method (Fig. 2). Moreover, although (3) contains a multiplication, since Δ is a very small integer (at most 9, see Table 1) for most cases, the multiplication can easily be converted to shift and add operations. For example, when $\Delta = 9$, $a_i \equiv T_i \ll 3 + T_i + x_i \pmod{n_{set}}$, where $\ll$ denotes a left shift operation. Finally, when a'_i is still large, we can apply (3) iteratively to obtain a''_i, a'''_i , etc. The following theorem states the maximum number of iterations that need to be performed to compute a cache index using the iterative linear method.

Theorem 1. Assuming a 2-input subtract&select unit and given a B -bit address and a cache with block/line size of L , the number of iterations needed to compute $a_i \pmod{n_{set}}$ is at most:

$$\left\lceil \frac{B - \log_2 L - \log_2 n_{set}}{\log_2 n_{set_phys} - \log_2 \Delta} \right\rceil.$$

When a subtract&select with $2^t + 2$ selector inputs is used in conjunction with the iterative linear method, the required number of iterations is at most:

$$\left\lceil \frac{B - \log_2 L - \log_2 n_{set}}{t + \log_2 n_{set_phys} - \log_2 \Delta} \right\rceil.$$

Proof. In order to have $a_i \pmod{n_{set}} = a_i$, we need to have $a_i < n_{set}$. We will look at how the maximum range of a_i can be increased using (3) while still avoiding the modulo operation. We express a_i as $a_i = n_{set} \cdot x_1 + y_1$, where $y_1 < n_{set}$, and $a_i = n_{set_phys} \cdot x_2 + y_2$ where $y_2 < n_{set_phys}$. Now, using (3), we have $a_i \equiv \Delta \cdot x_2 + y_2 \pmod{n_{set}}$. In order to avoid the modulo computation, $a_i = \Delta \cdot x_2 + y_2 < n_{set}$. Hence, $x_2 < \frac{n_{set} - y_2}{\Delta} < \frac{n_{set}}{\Delta}$. Therefore,

$$a_i = n_{set_phys} \cdot x_2 + y_2 < n_{set_phys} \cdot \frac{n_{set}}{\Delta} + y_2.$$

Since $y_2 < n_{set_phys}$, it can be tackled separately by using the subtract&select method with only two inputs. Thus, $a_i < n_{set} \cdot \frac{n_{set_phys}}{\Delta}$. Compared to the original range, where $a_i < n_{set}$, we have increased the range of a_i 's value by $\frac{n_{set_phys}}{\Delta}$ times. Using induction, we can prove that, after n iterations, the maximum range of a_i such that no modulo

computation is required is $a_i < n_{set} \left(\frac{n_{set_phys}}{\Delta} \right)^n$. For a B -bit address and a cache with block/line size of L , the maximum value of a_i is $2^{B - \log_2 L}$. Therefore, we need n iterations such that $2^{B - \log_2 L} < n_{set} \left(\frac{n_{set_phys}}{\Delta} \right)^n$. Taking the log for both sides, the inequality becomes:

$$B - \log_2 L < \log_2 n_{set} + n \cdot (\log_2 n_{set_phys} - \log_2 \Delta).$$

Equivalently, $n > \left\lceil \frac{B - \log_2 L - \log_2 n_{set}}{\log_2 n_{set_phys} - \log_2 \Delta} \right\rceil$. Using a similar method, we can prove the case where there are $2^t + 2$ inputs into the subtract&select unit. $\square$

For example, for a 32-bit machine with $n_{set_phys} = 2,048$ and a 64-byte cache line size, the prime modulo can be computed with only two iterations. However, with a 64-bit machine, it requires six iterations using a subtract&select with 3-input selector, but requires three iterations with a 258-input selector.

Polynomial method. Because, in some cases, the iterative linear method involves multiple iterations, we devise an algorithm to compute the prime modulo operation in one step. To achieve that, using the same method as in deriving (3), we first express a_i as a polynomial function of n_{set_phys} :

$$\begin{aligned}
 a_i &\equiv x_i + t_{i1} \cdot n_{set_phys} + t_{i2} \cdot n_{set_phys}^2 + \dots + t_{in} \cdot n_{set_phys}^n \\
 &\pmod{n_{set}},
 \end{aligned}$$

where t_{ij} consists of bit $\log_2 n_{set_phys} \cdot j$ through bit $\log_2 n_{set_phys} \cdot (j + 1) - 1$ of the block address bits of a_i . For example, t_{i1} is shown as t_i in Fig. 1. Substituting n_{set_phys} by $(n_{set} + \Delta)$, we obtain

$$\begin{aligned}
 a_i &\equiv x_i + t_{i1} \cdot (n_{set} + \Delta) + t_{i2} \cdot (n_{set} + \Delta)^2 + \dots \\
 &\quad + t_{in} \cdot (n_{set} + \Delta)^n \pmod{n_{set}},
 \end{aligned}$$

$(n_{set} + \Delta)^k$, where $k = 1, 2, \dots, n$, can be expanded into

$$\frac{k!}{k!0!} n_{set}^k + \frac{k!}{(k-1)!1!} n_{set}^{(k-1)} \Delta^1 + \dots + \Delta^k.$$

In the $(\pmod{n_{set}})$ space, any term that is a multiple of n_{set} is equivalent to zero. Since only the last term is not zero, then $(n_{set} + \Delta)^k \equiv \Delta^k \pmod{n_{set}}$. Therefore, we can express a_i as a polynomial function of Δ :

$$\begin{aligned}
 a_i &\equiv x_i + t_{i1} \cdot \Delta + t_{i2} \cdot \Delta^2 + \dots + t_{in} \cdot \Delta^n \pmod{n_{set}} \\
 &\equiv a_i^* \pmod{n_{set}}.
 \end{aligned} \tag{4}$$

Note that a_i^* is much smaller than a_i and is, in general, small enough to derive the result of the prime modulo using the subtract&select method (Fig. 2).

A special but restrictive case is when n_{set} is a Mersenne prime number, in which case, $\Delta = 1$. Then, (4) can be simplified further, leading up to the method given by Yang and Yang [25]:

$$a_i \equiv x_i + t_{i1} + t_{i2} + \dots + t_{in} \pmod{n_{set}}. \tag{5}$$

It is possible to use n_{set} that is equal to $n_{set_phys} - 1$ but not a prime number. In some cases, if $n_{set_phys} - 1$ is not a prime number, it is a product of a few prime numbers (e.g., $255 = 3 \times 5 \times 17$, $511 = 7 \times 73$, $1,023 = 3 \times 11 \times 31$, etc.). Thus, some of them may be good choices to handle small strided patterns. However, it is beyond the scope of this paper to evaluate such numbers.

$$\begin{array}{lcl}
 x = & x^{10} x^9 x^8 x^7 x^6 x^5 x^4 x^3 x^2 x^1 x^0 & \\
 9 t_1 = & t_1^{10} t_1^9 t_1^8 t_1^7 t_1^6 t_1^5 t_1^4 t_1^3 t_1^2 t_1^1 t_1^0 & \text{A} \\
 & t_1^{10} t_1^9 t_1^8 t_1^7 t_1^6 t_1^5 t_1^4 t_1^3 t_1^2 t_1^1 t_1^0 & \text{B} \\
 & t_1^7 t_1^6 t_1^5 t_1^4 t_1^3 t_1^2 t_1^1 t_1^0 t_1^{10} t_1^9 t_1^8 & \text{C} \\
 81 t_2 = & t_2^3 t_2^2 t_2^1 t_2^0 t_2^0 t_2^0 t_2^0 t_2^0 t_2^0 t_2^0 t_2^0 & \text{D} \\
 & t_2^3 t_2^2 t_2^1 t_2^0 t_2^0 t_2^0 t_2^0 t_2^0 t_2^0 t_2^0 t_2^0 & \text{E} \\
 \hline
 & \text{index (mod 2039)} + & \text{index (mod 2039)} +
 \end{array}$$

(a) (b)

Fig. 3. (a) The initial components of index calculation, and (b) the components after optimizations.

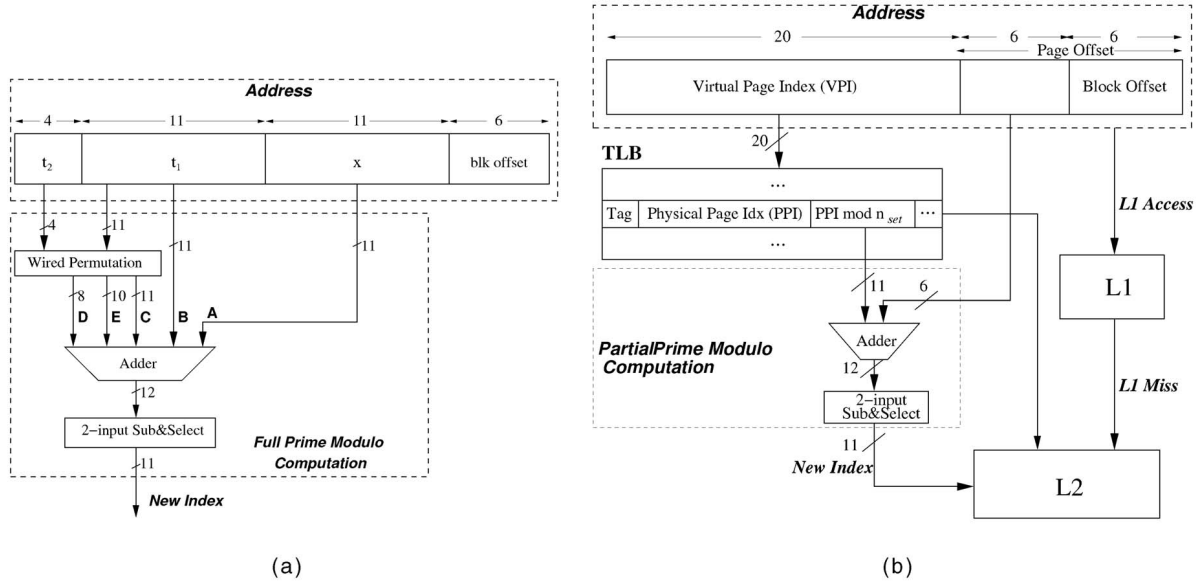


Fig. 4. Hardware implementation of the prime modulo hashing using the polynomial method for $n_{set} = 2,039$ on a 32-bit machine when (a) the full address is used to compute the new index and (b) when part of the address is precomputed and stored in the TLB.

Comparing the iterative linear and polynomial methods, the polynomial method allows smaller latency in computing the prime modulo when Δ is small, especially for 64-bit machines and a small number of sets in the cache. The iterative linear method is more desirable for low hardware and power budget, or when Δ is large.

3.1.1 Hardware Implementation

To illustrate how the prime modulo indexing can be implemented in hardware, let us consider an L2 cache with 64 byte blocks and 2,048 ($= 2^{11}$) number of physical sets and 2,039 ($= 2^{11} - 9$) number of sets. Therefore, 6 bits are used as the block offset, while the tag can be broken up into three components: 11-bit x ($x^{10}, x^9, x^8, \dots, x^0$), 11-bit t_1 ($t_1^{10}, t_1^9, t_1^8, \dots, t_1^0$), and the remaining 4-bit t_2 ($t_2^3, t_2^2, t_2^1, t_2^0$). According to (4), the cache index can be calculated as $\text{index} = x + 9 \cdot t_1 + 81 \cdot t_2 \pmod{n_{set}}$. The binary representations of 9 and 81 are “1001” and “1010001,” respectively. Therefore, Fig. 3a shows that the index can be calculated as the sum of six numbers. To simplify the computation further, the third number can be expressed as an addition of two numbers: one that consists of the three most significant bits ($t_1^{10} \cdot 2^{13} + t_1^9 \cdot 2^{12} + t_1^8 \cdot 2^{11}$) and the rest ($t_1^7 \cdot 2^{10} + t_1^6 \cdot 2^9 + \dots + t_1^0 \cdot 2^3$). The former number’s prime modulo can be computed separately and, according to (4), is

equal to $9 \cdot (t_1^{10} \cdot 2^2 + t_1^9 \cdot 2^1 + t_1^8 \cdot 2^0) = (t_1^{10} \cdot 2^5 + t_1^9 \cdot 2^4 + t_1^8 \cdot 2^3) + (t_1^{10} \cdot 2^2 + t_1^9 \cdot 2^1 + t_1^8 \cdot 2^0)$ in the modulo space. Furthermore, some of the numbers can be added instantly to fill in the bits that have zero values. For example, the fourth and the fifth numbers are combined into a single number. The resulting numbers are shown in Fig. 3b. There are only five numbers (A through E) that need to be added, with up to 11 bits each.

Fig. 4 shows how the prime modulo hashing computation fits with the overall system when the index from the full address is computed (Fig. 4a), or from part of the address (Fig. 4b). Fig. 4a shows how the *new index* is computed from the addition of the five numbers (A through E in Fig. 3b). A and B are directly taken from the address bits, while C, D, and E are obtained by wired permutation of the tag part of the address. The sum of the five numbers is then fed into a subtract&select unit. In order to keep the sum of the five numbers (A through E) and the number of inputs into the selector small, (4) can be used in the intermediate additions by converting any intermediate carry out (in the 11th most significant position) into a 9 and adding it to the intermediate addition results. This ensures that the overall sum can only be slightly larger than 2,039 and that only two inputs would be needed in the selector.

Fig. 4b illustrates how the prime modulo indexing is integrated in a real system with two optimizations. For

illustration purposes, the figure assumes a virtually indexed physically tagged L1 cache, a physically indexed and tagged L2 cache, and 4KB page sizes. A physical address consists of the Physical Page Index (PPI) and the page offset. In the first optimization, we overlap the prime modulo computation of the L2 cache index with L1 cache accesses. On each L1 cache access, the prime modulo L2 cache index is computed. If the access results in an L1 cache miss, the new L2 cache index has been computed and is ready for use. This completely hides the prime modulo computation latency, at the expense of higher power consumption.

In the second optimization, we reduce the computational complexity and power consumption of the prime modulo computation by separating the prime modulo computation of the PPI and the block address. On a TLB miss, $PPI \bmod n_{set}$ of the missed page is computed and, along with the address translation information, stored in the new TLB entry. This computation is not in the critical path of the TLB access and does not require modifications to the operating system's page table. To compute the L2 cache index, we simply add the precomputed PPI modulo with the page offset bits that are not part of the L2 block offset. In the figure, only 6 bits of the page offset need to be added to the 11-bit precomputed PPI modulo, followed by a 2-input subtract&select operation to obtain the L2 cache index. This is a very simple operation that can probably be performed in much less than one clock cycle with small power consumption.

3.2 The Odd-Multiplier Displacement Hashing Function

In the odd-multiplier displacement hashing, the traditional modulo is performed after an offset is added to the original index x_i . The offset is the product of a *multiplier* p and the tag T_i .⁴

$$H(a_i) = (p \cdot T_i + x_i) \bmod n_{set}. \quad (6)$$

This new hashing function is based on hashing functions in Aho and Ullman [1] and is related to Raghavan and Hayes's RANDOM-H functions [15], with the main difference being their use of nonconstant offset, which results in not satisfying the sequence invariance property. Fig. 5 shows the hardware implementation used in odd-multiplier displacement hashing for a 32-bit machine. The figure uses 9 as the multiplier. The multiplication between the tag and the multiplier is converted into a simple addition operation. Therefore, the index can be calculated with a truncated addition of three 11-bit numbers. In general, we can minimize the number of narrow truncated additions by choosing p with few 1s in its binary representation.

One advantage of the odd-multiplier displacement hashing function compared to the prime modulo hashing function is that the complexity of calculating the cache index in the odd-multiplier displacement hashing function is mostly independent of the machine size. This makes it trivial to implement in machines with 64-bit or larger addressing.

4. Our earlier work [9] refers to it as Prime Displacement Indexing because we originally only considered prime numbers for p , but found out that some odd numbers performed well.

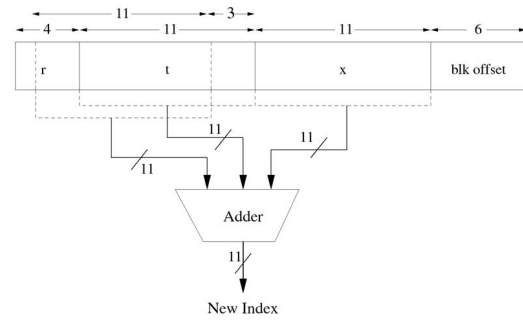


Fig. 5. Hardware implementation of the odd-multiplier displacement hashing for $n_{set} = 2,048$ on a 32-bit machine.

3.2.1 Choosing the Multiplier in Odd-Multiplier Displacement Hashing

An important issue in the odd-multiplier displacement hashing function is how to find a good multiplier. Since we found that most odd numbers can achieve an ideal balance for most stride amounts, we will focus on discussing the concentration of such numbers. Fig. 6 shows the concentration achieved using different multipliers for strided accesses with stride amounts ranging from 1 to 8. Only small stride amounts are considered because they are more common than larger ones. Each of the eight subfigures represents a different stride amount. The x-axis shows the different prime or odd-multipliers investigated. Discontinuity in the figure indicates a concentration value larger than 500. The figure shows that the concentration is usually better (lower) for multipliers. Some multipliers suffer from bad concentration for certain stride amounts. For example, 5 has a poor concentration when the stride amount is 3, while 3 has a poor concentration when the stride amount is 5.

A good multiplier should not only give good average concentration across all stride amounts, but also ensure good concentration for small stride amounts because they are more common than large stride amounts in practice. We found that the following multipliers excel in both criteria: 9, 21, 31, and 61.

3.3 Comparison of Various Hashing Functions

Before we compare our hashing functions with existing ones, we briefly overview existing hashing functions. The *traditional hashing* function H_{trad} is a very simple modulo-based hashing function. It can be expressed as $H_{trad} = x_i$ or, equivalently, as $H_{trad}(a_i) = a_i \bmod n_{set,phys}$. Pseudorandom hashing randomizes accesses to cache sets. Examples of pseudorandom hashing are XOR-based hashing functions, which are by far the most extensively studied [16], [17], [22], [23], [26], [12], [18], [4], [19]. We choose one of the most prominent examples: $H(a_i) = t_i \oplus x_i$, where $\oplus$ represents the bitwise exclusive OR operator. In a *skewed associative cache*, the cache itself is divided into banks and each bank is accessed using a different hashing function. Here, cache blocks that are mapped to the same set in one bank are most likely not to map to the same set in the other banks. Seznec and Bodin propose using an XOR hashing function in each bank after a circular shift is performed to the bits in t_i [18], [4], [19]. The number of circular shifts performed differs between banks. This results in a form of a perfect shuffle.

We propose using odd-multiplier displacement hashing

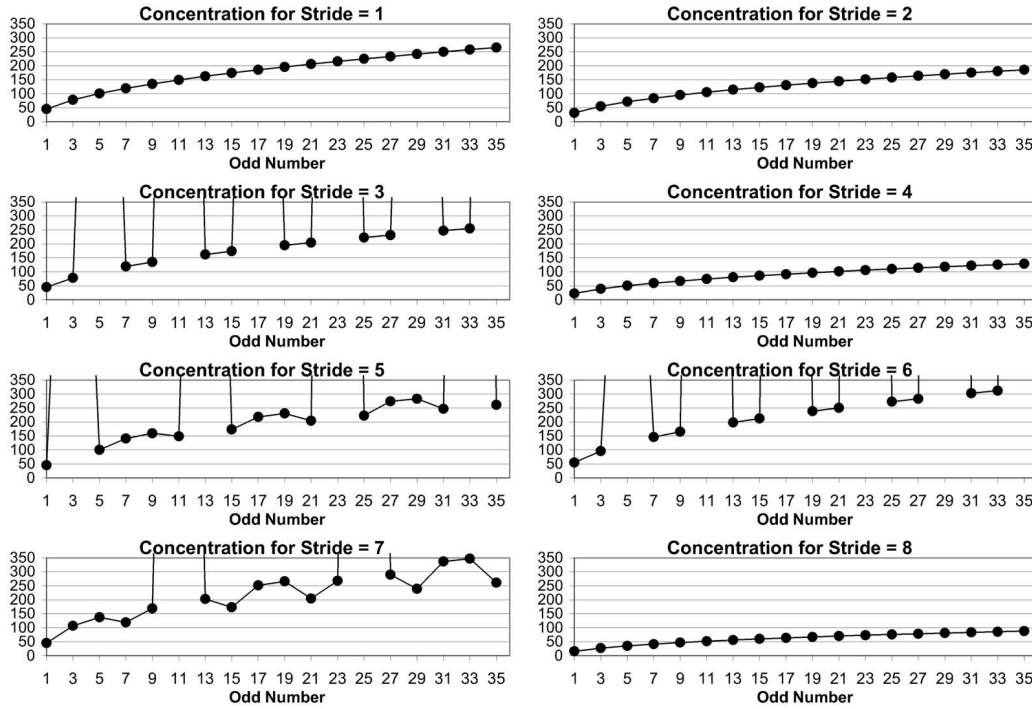


Fig. 6. Concentration obtained using different multipliers for strided accesses with different stride values (1-8).

TABLE 2
Qualitative Comparison of Various Hashing Functions: Traditional, XOR, Prime Modulo (pMod), Odd-Multiplier Displacement (oDisp), Skewed Associative with Circular-Shift and XOR (Skewed) [18], [4], [19], and Our Skewed Associative with Odd-Multiplier Displacement (Skewed + oDisp)

Characteristics	Single Hashing Function				Multiple Hashing Functions	
	Traditional	XOR	pMod	oDisp	Skewed	Skewed + oDisp
Ideal balance condition	s odd	various	all s except $s = k \times n_{set}$	Most odd, all even s	None	None
Sequence invariant?	Yes	No	Yes	Partial	No	No
Simple Hw Impl.	Yes	Yes	Yes	Yes	Yes	Yes
Replacement Alg. Restriction	No	No	No	No	Yes	Yes

functions with a skewed cache. To ensure interbank dispersion, a different multiplier for each bank is used.

3.3.1 Qualitative Comparison

Table 2 compares the various hashing functions based on when the ideal balance is achieved, whether they satisfy the *sequence invariance* property, whether a simple hardware implementation exists, and whether they place restrictions on the replacement algorithm. The major disadvantage of the traditional hashing is that it achieves the ideal balance only when the stride amount s is odd, where $\gcd(s, n_{set}) = 1$. When the ideal balance is satisfied, however, it achieves the ideal concentration because it satisfies the sequence invariance property. Note that, for a frequent case of a unit stride ± 1 , it has an ideal balance and concentration. Thus, any hashing functions that achieve less than the ideal balance or concentration with unit strides are prone to exhibit a pathological behavior.

XOR achieves the ideal balance on most stride amounts s , but always has less than the ideal concentration because it

does not satisfy the sequence invariance property. This is because the sequence of set accesses is never repeated due to XOR-ing with different values in each subsequence. Thus, it is prone to pathological behavior. There are various cases where the XOR hashing does not achieve the ideal balance. One case is when $s = n_{set} - 1$. For example, with $s = 15$ and $n_{set} = 16$ (as in a 4-way 4KB cache with 64 byte lines), it will access sets 0, 15, 15, 15, ... Not only that, a stride of 3 or 5 will also fail to achieve the ideal balance because they are factors of 15. This makes XOR hashing a particularly bad choice for indexing the L1 cache.

The prime modulo hashing (*pMod*) achieves the ideal balance and concentration except for very few cases. The ideal balance is achieved because $\gcd(s, n_{set}) = 1$ except when s is a multiple of n_{set} . If $a_i = a_{i+d} \pmod{p}$ for some d , then $a_i + s = a_{i+d} + s \pmod{p}$, for any s . Therefore, with prime modulo hashing, the ideal concentration is achieved because $H(a_i) = H(a_{i+d})$ implies that $H(a_{i+1}) = H(a_i + s) = H(a_{i+d} + s) = H(a_{i+d+1})$ with the stride amount s . This makes the prime modulo hashing an ideal hashing function.

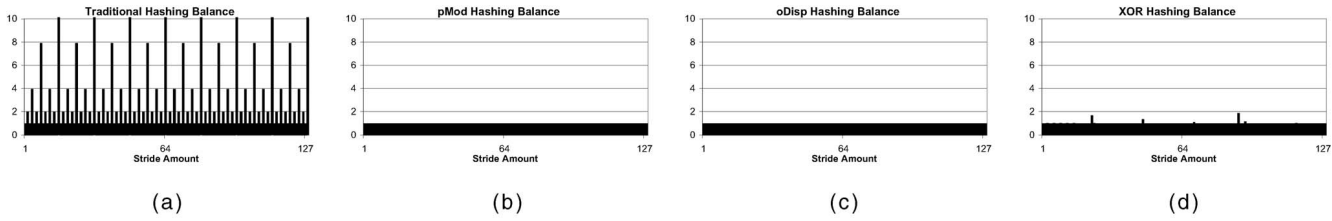


Fig. 7. Balance: (a) traditional Hashing, (b) prime modulo hashing, (c) odd-multiplier displacement Hashing, and (d) XOR hashing.

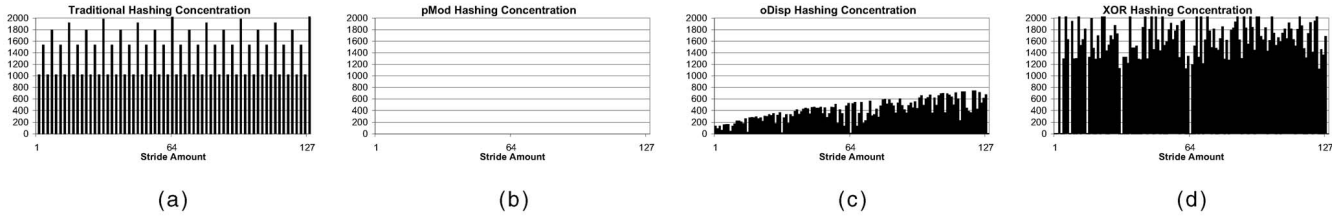


Fig. 8. Concentration: (a) traditional hashing, (b) prime modulo hashing, (c) odd-multiplier displacement hashing, and (d) XOR hashing.

As we have shown in Section 3.1, fast hardware implementations exist.

The odd-multiplier displacement hashing (*oDisp*) achieves an ideal balance with even strides and most odd strides. Although it does not satisfy the sequence invariance property, the distance between two accesses to the same set is almost always constant. That is, for all but one set in a single subsequence, $H(a_i) = H(a_{i+x})$ implies $H(a_{i+1}) = H(a_{i+x+1})$. Furthermore, $x = n_{set} - p$, where p is the multiplier. Thus, it partially satisfies the sequence invariance.

Skewed associative caches do not guarantee the ideal balance or concentration, whether XOR-based hashing (*Skewed*) or odd-multiplier displacement hashing (*Skewed+oDisp*) is used. However, probabilistically, the accesses will be quite balanced since an address can be mapped to a different place in each bank. A disadvantage of skewed associative caches is the fact that they make it difficult to implement a least recently used (LRU) replacement policy and force using pseudo-LRU policies. The nonideal balance and concentration, together with the use of a pseudo-LRU replacement policy, make a skewed associative cache prone to pathological behavior, although it works well on average. Later, we will show that skewed caches do degrade the performance of some applications.

Finally, although not described in Table 2, some other hashing functions, such as all XOR-based functions and random-h [15], [16], [17], [20], [8], [7], [26], [12], are not sequence invariant and, therefore, do not achieve the ideal concentration.

3.3.2 Balance and Concentration

Fig. 7 shows the balance values of the four different hashing functions using a synthetic benchmark that produces only strided access patterns. The stride size is varied from 1 to 128. The maximum balance and concentration displayed in the vertical axes in the figures are truncated at 10 and 2,000, respectively, for easy comparison. Note that, since small strides are more likely to appear in practice, they are more important than large strides. The balance values for the traditional and prime modulo hashing functions follow the discussion in Section 3.3. In particular, the traditional hashing function suffers from bad balance values with even strides, but achieves a perfect balance with odd

strides. The prime modulo achieves perfect balance, except when the stride is equal to n_{set} , which, in this case, is 2,039. XOR and odd-multiplier displacement hashing also have the ideal balance with most strides. Both have various cases in which the stride size causes nonideal balance. However, the odd-multiplier displacement hashing achieves ideal balance for the smaller strides, making it, in practice, superior to the XOR hashing function.

Fig. 8 shows the concentration for the same range of stride amounts. As expected, the traditional hashing function suffers from very bad concentration with even strides, but achieves perfect concentration with odd strides. The XOR and odd-multiplier displacement hashing functions also suffer from bad concentration for many strides. This is because odd-multiplier displacement hashing is only partially sequence invariant. However, note that odd-multiplier displacement hashing achieves better concentration on small stride amounts compared to XOR hashing. We can also expect that, in an application with only small stride amounts, the odd-multiplier displacement algorithm will perform well. Since the prime modulo hashing is sequence invariant, it achieves ideal concentration except for the stride equal to n_{set} . Hence, for strided accesses, we can expect the prime modulo hashing to have the best performance between our four hashing functions.

More importantly, the prime modulo hashing also achieves ideal concentration with odd strides the same way as the traditional hashing. Hence, we can expect that the prime modulo hashing is resistant to the pathological behavior. The XOR hashing function may outperform the traditional hashing on average. However, it cannot match the ideal concentration of the traditional hashing with odd stride amount. Thus, it is prone to pathological behavior.

4 EVALUATION ENVIRONMENT

Applications. To evaluate the prime hashing functions, we use 23 memory-intensive applications from various sources: bzip2, gap, mcf, and parser from Specint2000 [21], applu, mgrid, swim, equake, and tomcatv from Specfp2000 and Specfp95 [21], mst from Olden, bt, ft, lu, is, sp, and cg from NAS [13], sparse from Sparsebench [6], and tree from the University of Hawaii [3]. Irr is an iterative PDE solver used

TABLE 3
L2 Cache Miss Rates for the Different Applications

applu	bt	bzip2	cg	charmm	equake	euler	ft	gap	irr	is	lu
42.7%	9.0%	23.1%	30.3%	7.4%	41.1%	6.5%	10.6%	48.6%	3.2%	58.5%	40.7%
mcf	mgrid	moldyn	mst	nbf	parser	sparse	sp	swim	tomcatv	tree	
15.2%	27.5%	2.0%	37.8%	48.8%	10.0%	28.5%	35.0%	35.9%	19.6%	20.0%	

TABLE 4
Parameters of the Simulated Architecture

PROCESSOR
6-issue dynamic. 1.6 GHz. Int, fp, ld/st FUs: 4, 4, 2 Pending ld, st: 8, 16. Branch penalty: 12 cycles
MEMORY
L1 data: write-back, 16 KB, 2 way, 32-B line, 3-cycle hit RT, lockup free, 24+24 pending ld&st misses, LRU L2 data: write-back, 512 KB, 4 way, 64-B line, 16-cycle hit RT, lockup free, 24+24 pending ld&st misses, LRU RT memory latency: 243 cycles (row miss), 208 cycles (row hit) Memory bus: split-transaction, 8 B, 400 MHz, 3.2 GB/sec peak Dual channel DRAM. Each channel: 2 B, 800 MHz. Total: 3.2 GB/sec peak Random access time (t_{RAC}): 45 ns Time from memory controller (t_{System}): 60 ns

Latencies correspond to contention-free conditions. RT stands for round-trip from the processor.

in CFD applications. Charmm is a well-known molecular dynamics code and moldyn is its kernel. Nbf is a kernel from the GROMOS molecular dynamics benchmarks. And, euler is a 3D Euler equation solver from NASA.

We categorize the applications into two groups: a group where the histogram of the number of accesses to different sets is uniform and a group where it is not uniform. Let $f_1, f_2, \dots, f_{n_{set}}$ represent the frequency of accesses to the sets $1, 2, \dots, n_{set}$ in the L2 cache. An application is considered to have a nonuniform cache access behavior if the ratio $stdev(f_i)/\bar{f}$ is greater than 0.5. Applications with nonuniform cache set accesses likely suffer from conflict misses and, hence, alternative hashing functions are expected to speed them up.

Among the 23 applications, we found that 30 percent of them (seven benchmarks) are nonuniform: bt, cg, ft, irr, mcf, sp, and tree. The L2 cache miss rates for all the applications are summarized in Table 3.

Simulation Environment. The evaluation is performed using a cycle accurate execution-driven simulation environment that supports a dynamic superscalar processor model [10]. Table 4 shows the parameters used for each component of the architecture. The latency of prime modulo and odd-multiplier displacement computation is assumed to be less than three cycles and, therefore, can be fully overlapped with the L1 cache access latency and does not cause any addition to the L2 cache access time (Section 3.1.1).

Prime Numbers. The prime modulo function uses the prime number shown in Table 1. The odd-multiplier displacement function uses a number 9 when it is used as a single hashing function. When used in conjunction with a skewed associative cache the multipliers for each of the four cache banks are 9, 19, 31, and 37.

5 EVALUATION

In this section, we present and discuss five sets of evaluation results. We present the impact of using different single hashing functions on the various types of cache misses (Section 5.1) and on the cache miss distribution

across the cache sets (Section 5.2). Section 5.3 presents the impact of using multiple hashing functions in conjunction with a skewed associative L2 cache on the various types of cache misses. Section 5.4 shows the performance gain achieved using the different hashing functions for the 23 applications. Finally, Section 5.5 shows the impact of our prime hashing functions on the execution time for various cache configurations.

5.1 Single Hashing Function Schemes

Figs. 9 and 10 show the normalized number of cache misses in each application with nonuniform cache accesses and uniform cache accesses, respectively. Each figure compares the number of cache misses with different hashing functions: a traditional hashing function with 4-way associative L2 cache (*Base*), a traditional hashing function with an 8-way associative same-size L2 cache (*8-way*), the XOR hashing function (*XOR*), the prime modulo hashing function (*pMod*), and the odd-multiplier displacement hashing function (*oDisp*), as described in Section 3.3. The number of misses in each case is normalized to *Base*. All bars are divided into: cold misses (*Cold*), capacity misses (*Capacity*), and conflict misses (*Conflict*). The cold misses are computed as the sum of the first miss to each line that is accessed. The conflict misses are calculated as the number of misses eliminated if a fully associative L2 cache is used. The rest of the misses are categorized as capacity misses.

Fig. 9 shows that, for nonuniform applications, conflict misses account for roughly 50 percent of the total L2 cache misses. Fig. 10 shows that even uniform applications suffer from a noticeable amount of conflict misses, averaging 17 percent of the total L2 cache misses.

For nonuniform applications, Fig. 9 shows that increasing the L2 cache associativity to 8-way only reduces the number of misses marginally (10 percent on average). This is mainly because doubling associativity but keeping the same cache size reduces the number of sets in half. In turn, this doubles the number of addresses mapped to a set. Thus, increasing cache associativity without increasing the cache size is not an effective method to eliminate conflict

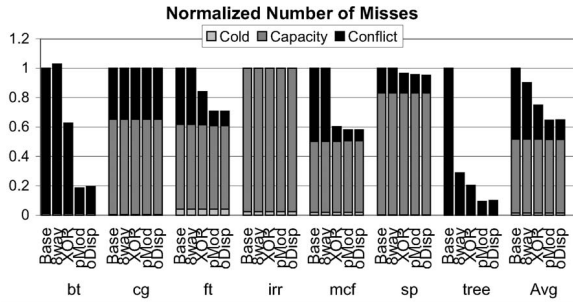


Fig. 9. Reduction in cache misses of single hashing functions for applications with nonuniform cache accesses.

misses. Comparing the alternative hashing functions, *pMod* and *oDisp* perform the best, eliminating a large majority of L2 conflict misses (70 percent on average), contributing to a 35 percent total L2 cache miss reduction. XOR is much less effective, on average only eliminating 25 percent of the L2 cache misses. Although XOR is certainly better than *Base* and *8-way*, its nonideal balance for small strides and its nonideal concentration hurt its performance.

For applications that have uniform cache accesses in Fig. 10, the same observation generally holds. *pMod* and *oDisp* noticeably reduce the number of conflict misses in charmm, euler, and mst.

5.2 Cache Misses Distribution

Fig. 11 shows the cache misses distribution over the cache sets for the nonuniform applications using three different hashing functions: traditional (*Base*), prime modulo (*pMod*), and odd-multiplier displacement (*oDisp*). The x-axis represents the cache sets, while the y-axis represents the number of cache misses associated with that set. The figure shows how the distribution changes as the result of applying the *pMod* and *oDisp* hashing functions.

The figure shows that *bt* and *tree* have the worst distribution of cache misses using *traditional* hashing and exhibit a much better distribution and, consequently, a large reduction in the number of misses when *pMod* or *oDisp* hashing is used. For *tree*, with the traditional hashing function, the vast majority of cache misses are concentrated in about 10 percent of the sets. This is due to an unbalanced distribution of cache accesses, causing some cache sets to be overutilized and, thus, to suffer from many conflict misses. By distributing the accesses more uniformly across the sets, *pMod* and *oDisp* are able to eliminate most of those misses. Other applications (*mcf*, *sp*, and *ft*) also have a bad distribution when

traditional hashing is used, but significantly improved distributions when *pMod* or *oDisp* hashing functions are used.

5.3 Multiple Hashing Functions Schemes

Figs. 12 and 13 show the normalized number of misses in applications with nonuniform cache accesses and uniform cache accesses, respectively, when multiple hashing functions are used. Each figure compares the number of misses with different hashing functions: a traditional hashing function with 4-way associative L2 cache (*Base*), the prime modulo hashing that is the best single hashing function from Section 5.1 (*pMod*), the XOR-based skewed associative cache proposed by Seznec [19] (*SKW*), and the skewed associative cache with the odd-multiplier displacement function that we propose (*skw+oDisp*) as described in Section 3.3. The number of misses in each case is normalized to *Base*.

The skewed associative caches (*SKW* and *skw+oDisp*) are based on Seznec's design that uses four direct-mapped cache banks. The replacement policy is called Enhanced Not Recently Used (ENRU) [19]. The only difference between *SKW* and *skw+oDisp* is the hashing function used (XOR versus odd-multiplier displacement). We have also tried a different replacement policy, called NRUNRW (Not Recently Used Not Recently Written) [18]. We found that it gives similar results.

Figs. 12 and 13 show that *pMod* sometimes outperforms and is sometimes outperformed by *SKW* and *skw+oDisp*. Interestingly, when skewed caches (*SKW* and *skw+oDisp*) outperform (*pMod*), it is mostly due to the reduction in *capacity misses* instead of in *conflict misses* (see *cg* and *mst*). Indeed, in terms of conflict miss reduction, *pMod* appears to be superior. It is more effective in eliminating conflict misses in *ft*, *mcf*, *charmm*, and *euler*, but is less effective in *bt*. However, *pMod* consistently reduces or maintains the conflict misses, and never increases them. In contrast, *SKW* and *skw+oDisp* actually increase the conflict misses in seven applications (*bzip2*, *charmm*, *is*, *mgrid*, *parser*, *sparse*, and *irr*) by up to 19 percent. On average, however, *skw+oDisp* performs the best, followed by *SKW* and then followed closely by *pMod*. In terms of average execution time speedups over *Base* on nonuniform applications, *skw+oDisp* achieves 1.35, followed by *SKW* (1.31) and *pMod* (1.27), as we will see in Section 5.4. The extra performance of skewed caches for the applications with nonuniform accesses, however, comes at the expense of having pathological behavior in applications that have uniform accesses (Fig. 13).

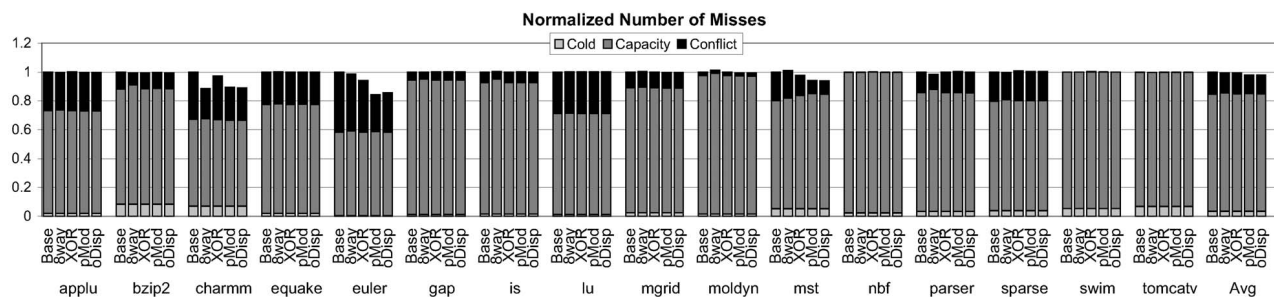


Fig. 10. Reduction in cache misses of single hashing functions for applications with uniform cache accesses.

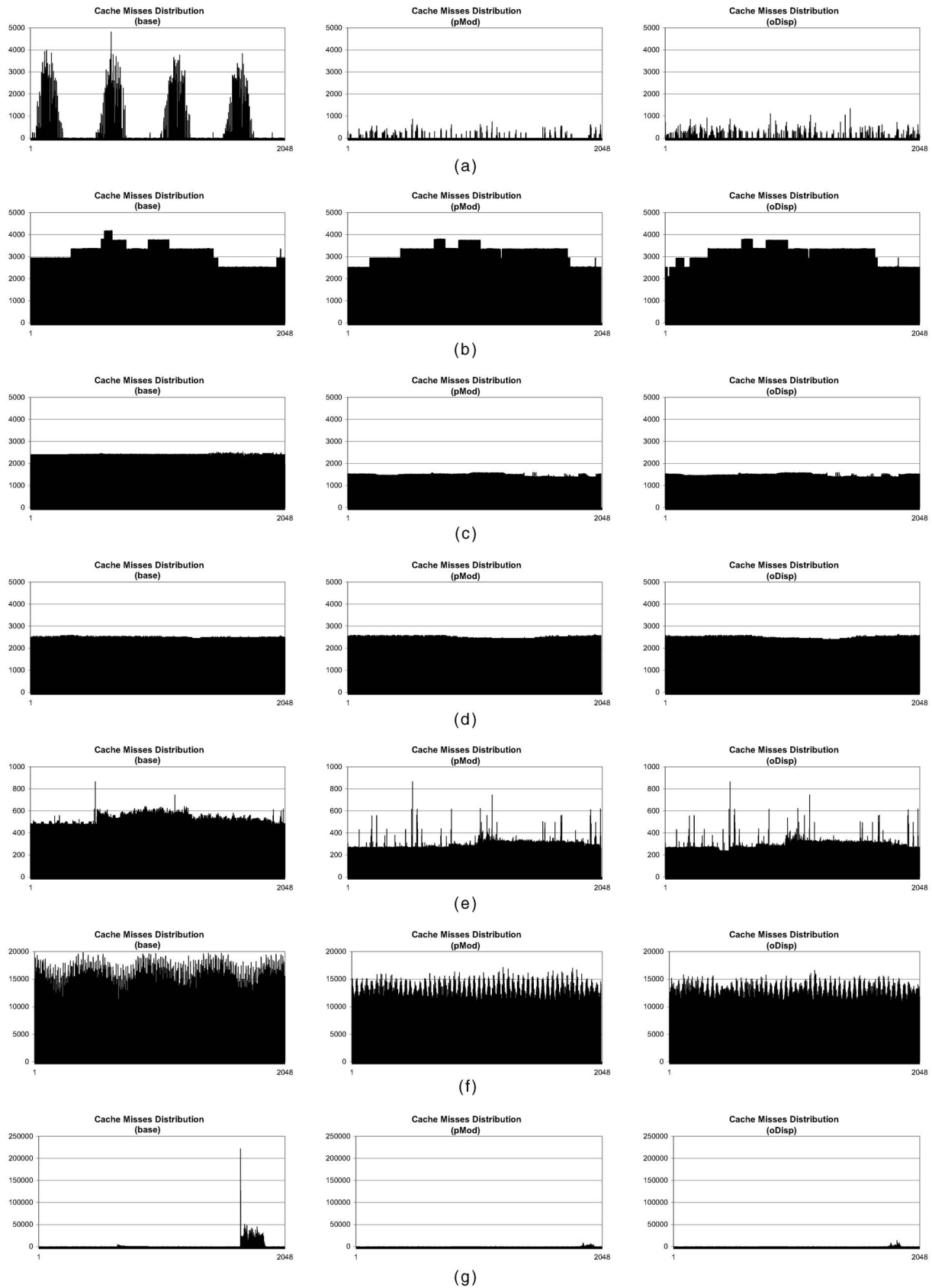


Fig. 11. Cache misses distribution for traditional, pMod, and oDisp hashing for nonuniform applications: (a) bt, (b) cg, (c) ft, (d) irr, (e) mcf, (f) sp, and (g) tree.

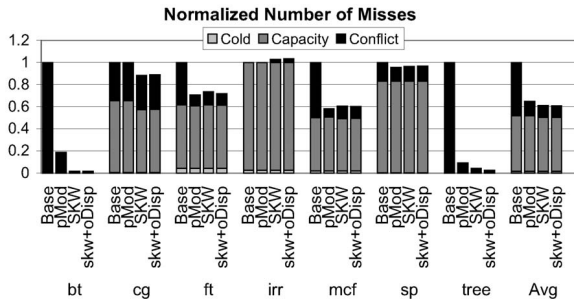


Fig. 12. Reduction in cache misses of multiple hashing functions for applications with nonuniform cache accesses.

5.4 Performance Improvement of the Different Schemes

Table 5 shows the performance improvement achieved for both the uniform and nonuniform applications using the five hashing functions: *XOR*, *pMod*, *oDisp*, *SKW*, and *skw+oDisp*. Speedups are calculated as the execution time improvement over traditional hashing (*Base*). In terms of average speedup, our *skw+oDisp* achieves the highest average (1.35) for nonuniform applications. *SKW* achieves an average speedup of 1.31, closely followed by *pMod* and *oDisp*, which achieve an average speedup of 1.27 each. The five hashing functions achieve their maximum improvement for *tree*. *skw+oDisp*

achieves a speedup of 2.63. *SKW*, *pMod*, and *oDisp* achieve comparable speedups of 2.55, 2.34, and 2.32, respectively. The higher performance gain achieved by the skewed caches comes at the expense of some pathological behavior in uniform applications. *SKW* slows down four applications (*bzip2*, *charmm*, *parser*, and *sparse*) by up to 9 percent for *sparse*. Our *skw+oDisp*, on the other hand, also slows down four applications (*bzip2*, *mgrid*, *parser*, and *sparse*) by up to 7 percent for *sparse*, but achieves better average performance for both uniform and nonuniform applications compared to *SKW* hashing. *pMod* only slows down one application (*sparse*) by 2 percent, while our proposed *oDisp* does not slowdown any of the 23 applications.

5.5 Sensitivity to Cache Parameters

This section evaluates the impact of using the prime hashing functions on the execution times of applications using different cache configurations. We choose only the best single hashing function (*pMod*) and the best multiple hashing functions (*skw+oDisp*). Tables 6 and 7 summarize the speedups on various cache configurations for *pMod* and *skw+oDisp* over *Base*, respectively. The cache size was varied from 256 KBytes to 1,024 KBytes while keeping the associativity constant at 4-way. The cache associativity was also varied from 2-way to 8-way while keeping the cache size constant at 512 KBytes. Both tables present the minimum, average, and maximum speedups obtained for

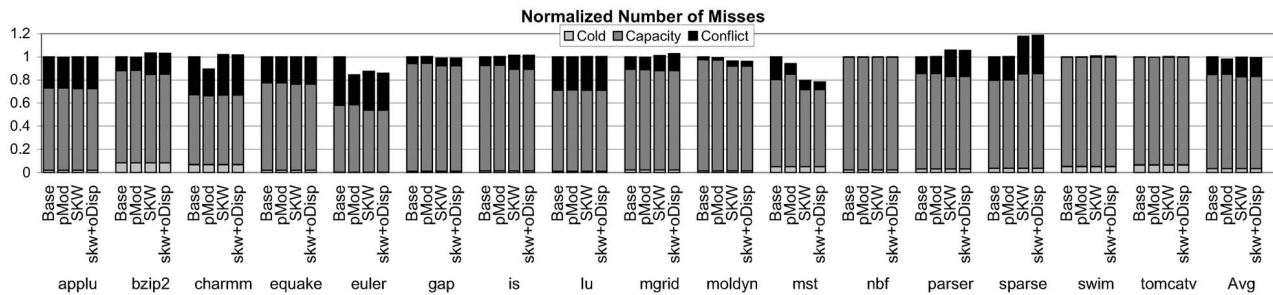


Fig. 13. Reduction in cache misses of multiple hashing functions for applications with uniform cache accesses.

TABLE 5
Summary of the Performance Improvement (Speedups) of the Different Hashing Functions for Uniform and Nonuniform Applications

Uniform Application	XOR	pMod	oDisp	SKW	skw+oDisp
applu	1.00	1.00	1.00	1.01	1.01
bzip2	1.00	1.00	1.00	0.97	0.95
charmm	1.01	1.04	1.04	0.98	1.02
equake	1.00	1.00	1.00	1.00	1.01
euler	1.01	1.05	1.05	1.02	1.08
gap	1.00	1.00	1.00	1.01	1.01
is	1.00	1.00	1.00	0.99	1.04
lu	1.00	1.00	1.00	0.99	0.99
mgrid	1.00	1.00	1.00	0.99	0.97
moldyn	1.00	1.00	1.00	1.00	1.00
mst	1.01	1.03	1.03	1.12	1.12
nbf	1.00	1.00	1.00	1.00	1.00
parser	1.00	1.00	1.00	0.98	0.97
sparse	0.98	0.98	1.00	0.91	0.93
swim	1.00	1.00	1.00	1.00	1.00
tomcatv	1.00	1.00	1.00	0.99	1.00
average	1.00	1.01	1.01	1.00	1.01

Nonuniform Application	XOR	pMod	oDisp	SKW	skw+oDisp
bt	1.05	1.13	1.13	1.17	1.17
cg	1.00	1.00	1.00	1.10	1.18
ft	1.09	1.15	1.16	1.12	1.15
irr	1.00	1.00	1.00	0.99	1.00
mcf	1.24	1.26	1.26	1.22	1.29
sp	1.02	1.02	1.02	1.01	1.00
tree	2.08	2.34	2.32	2.55	2.63
average	1.21	1.27	1.27	1.31	1.35

TABLE 6
Summary of the Performance Improvement (Speedups)
of the Different Cache Configurations
Using the Prime Modulo Hashing Function

Cache Configuration	Uniform Apps min,avg,max	Non-uniform Apps min,avg,max	Patho. Cases
256KB, 4-way	0.98,1.00,1.02	1.00,1.26,2.34	1
512KB, 4-way	0.98,1.01,1.05	1.00,1.27,2.34	1
1024KB, 4-way	0.98,1.01,1.10	0.99,1.19,1.84	1
512KB, 2-way	0.97,1.01,1.06	1.00,1.22,2.10	1
512KB, 4-way	0.98,1.01,1.05	1.00,1.27,2.34	1
512KB, 8-way	0.98,1.00,1.03	1.00,1.15,1.43	1

both uniform and nonuniform applications, and show the number of pathological cases that result. Pathological cases are defined as a slowdown of more than 1 percent compared to *Base*.

Table 6 shows that *pMod* produces good speedups for nonuniform applications on all cache configurations, with an average ranging from 1.15 to 1.27. These speedups are achieved without slowing down the uniform applications by more than 3 percent. As a result, the number of applications that exhibit pathological behavior are very limited (at most one case).

Table 7 shows that *skw+oDisp* produces higher speedups for nonuniform applications on all cache configurations, with an average ranging from 1.19 to 1.35. These average speedups are consistently higher than *pMod*, due to the ability of *skw+oDisp* to eliminate some capacity misses. However, the higher average speedups come at the expense of having many more pathological cases, mostly on uniform applications. Some applications exhibit nontrivial slowdown of up to 10 percent due to an increase in conflict misses. Consequently, the number of applications that exhibit pathological behavior is significantly higher, ranging from three to six.

6 RELATED WORK

Prior studies showed that alternative cache indexing/hashing functions are effective in reducing conflicts by achieving a more uniform access distribution across the L1 cache sets [18], [4], [23], [22], L2 cache sets [19], or the main memory banks [15], [11], [16], [17], [20], [8], [7], [26], [12]. Most of the prior hashing functions permute the accesses using some form of XOR operations. We found that XOR operations typically do not achieve an ideal concentration that is critical to avoiding pathological behavior under strided access patterns.

Although prime-based hashing has been proposed for software hash tables, as in [1], its use in the memory subsystem has been very limited due to its hardware complexity that involves true integer division operations and fragmentation problems. Budnick and Kuck first suggested using a prime number of memory banks in parallel computers [5], which was later developed into the Burroughs Scientific Processor [11]. Yang and Yang proposed using Mersenne prime modulo hashing for cache indexing for vector computation [25]. Since Mersenne prime numbers are sparse, e.g.,

$$31(2^5 - 1), 127(2^7 - 1), 8,191(2^{13} - 1), 131,071(2^{17} - 1), \dots,$$

TABLE 7
Summary of the Performance Improvement of the Different
Odd-Multiplier Displacement Skewed Cache Configurations

Cache Configuration	Uniform Apps min,avg,max	Non-uniform Apps min,avg,max	Patho. Cases
256KB, 4-way	0.90,1.03,1.42	0.94,1.31,2.77	6
512KB, 4-way	0.93,1.01,1.12	1.00,1.35,2.63	4
1024KB, 4-way	0.96,1.01,1.10	1.00,1.26,1.85	3
512KB, 2-way	0.94,1.00,1.05	0.98,1.31,2.45	5
512KB, 4-way	0.93,1.01,1.12	1.00,1.35,2.63	4
512KB, 8-way	0.93,1.01,1.13	1.00,1.19,1.48	5

using them significantly restricts the number of cache sets that can be implemented. We derive a more general solution that does not assume Mersenne prime numbers and show that prime modulo hashing can be implemented fast on *any* number of cache sets. In addition, we present an odd-multiplier displacement hashing function that achieves comparable performance and robustness to the prime modulo hashing function.

Compiler optimizations have also targeted conflict misses by padding the data structures of a program. One example is the work by Bacon et al. [2], who tried to find the optimal padding amount to reduce conflict misses in caches and TLBs in a loop nest. They tried to spread cache misses uniformly across loop iterations based on profiling information. Since the conflict behavior is often input dependent and determined at runtime, their approach has limited applicability.

7 CONCLUSIONS

Even though using alternative cache indexing/hashing functions is a popular technique to reduce conflict misses by achieving a more uniform cache access distribution across the sets in the cache, no prior study has really analyzed the pathological behavior of such hashing functions that often result in performance degradation.

We presented an in-depth analysis of the pathological behavior of hashing functions and proposed two new hashing functions for the L2 cache that are resistant to the pathological behavior and yet are able to eliminate the worst-case conflict behavior. The prime modulo hashing uses a prime number of sets in the cache, while the odd-multiplier displacement hashing adds an offset that is equal to a odd/prime number multiplied by the tag bits to the index bits of an address to obtain a new cache index. These hashing techniques can be implemented in fast hardware that uses a set of narrow add operations in place of true integer division and multiplication. This implementation has negligible fragmentation for the L2 cache. We evaluated our techniques with 23 applications from various sources. For applications that have nonuniform cache accesses, both the prime modulo and odd-multiplier displacement hashing achieve an average speedup of 1.27, practically without slowing down any of the 23 benchmarks.

Although lacking the theoretical superiority of the prime modulo, when the multiplier is carefully selected, the odd-multiplier displacement hashing performs just as well in practice. In addition, the odd-multiplier displacement hashing can easily be used in conjunction with a skewed associative cache, which uses multiple hashing functions to further distribute the cache accesses across the sets in the cache. The odd-multiplier displacement hashing

outperforms XOR-based hashing used in prior skewed associative caches. Although the skewed associative L2 cache with odd-multiplier displacement hashing eliminates less conflict misses than single hashing functions, it outperforms single hashing functions due to eliminating some capacity misses. It shows an average speedup of 1.35 for applications that have nonuniform cache accesses. However, it introduces some pathological behavior that slows down four applications by up to 7 percent. Therefore, an L2 cache with our prime modulo or odd-multiplier displacement hashing functions is a promising alternative.

ACKNOWLEDGMENTS

This work is supported in part by the US National Science Foundation through Faculty Early Career Development Award CCF-0347425 and North Carolina State University; the Ministry of Education under the Brain Korea 21 Project, by the Ministry of Science and Technology under the National Research Laboratory program, by the Institute of Computer Technology at Seoul National University, and by the IT SoC Promotion Group of the Korean IT Industry Promotion Agency under the Human Resource Development Project for IT SoC Key Architects.

REFERENCES

- [1] A.V. Aho and J.D. Ullman, *Principles of Compiler Design*, chapter 7.6, pp. 434-438. Addison-Wesley, 1997.
- [2] D.F. Bacon, J.-H. Chow, D. Ching, R. Ju, K. Muthukumar, and V. Sarkar, "A Compiler Framework for Restructuring Data Declarations to Enhance Cache and TLB Effectiveness," *Proc. 1994 Conf. Centre for Advanced Studies on Collaborative Research*, Oct. 1994.
- [3] J.E. Barnes, "Treecode," Inst. for Astronomy, Univ. of Hawaii, 1994, [ftp://hubble.ifa.hawaii.edu/pub/barnes/treecode](http://hubble.ifa.hawaii.edu/pub/barnes/treecode).
- [4] F. Bodin and A. Seznec, "Skewed-Associativity Improves Performance and Enhances Predictability," *IEEE Trans. Computers*, vol. 46, 1997.
- [5] P. Budnick and D.J. Kuck, "Organization and Use of Parallel Memories," *IEEE Trans. Computers*, vol. 20, no. 12, Dec. 1971.
- [6] J. Dongarra, V. Eijkhout, and H. van der Vorst, "SparseBench: A Sparse Iterative Benchmark," <http://www.netlib.org/benchmark/sparsebench>, 2005.
- [7] J.M. Frailong, W. Jalby, and J. Lenfant, "XOR-Schemes: A Flexible Data Organization in Parallel Memories," *Proc. Int'l Conf. Parallel Processing*, 1985.
- [8] D.T. Harper III and J.R. Jump, "Vector Access Performance in Parallel Memories Using a Skewed Storage Scheme," *IEEE Trans. Computers*, vol. 36, no. 12, Dec. 1987.
- [9] M. Kharbutli, K. Irwin, Y. Solihin, and J. Lee, "Using Prime Numbers for Cache Indexing to Eliminate Conflict Misses," *Proc. Int'l Symp. High Performance Computer Architecture*, 2004.
- [10] V. Krishnan and J. Torrellas, "A Direct-Execution Framework for Fast and Accurate Simulation of Superscalar Processors," *Proc. Int'l Conf. Parallel Architectures and Compilation Techniques*, Oct. 1998.
- [11] D.H. Lawrie and C.R. Vora, "The Prime Memory System for Array Access," *IEEE Trans. Computers*, vol. 31, no. 5, May 1982.
- [12] W.F. Lin, S.K. Reinhardt, and D.C. Burger, "Reducing DRAM Latencies with a Highly Integrated Memory Hierarchy Design," *Proc. Int'l Symp. High-Performance Computer Architecture*, 2001.
- [13] NAS Parallel Benchmark, <http://www.nas.nasa.gov/pubs/techreports/nasreports/nas-98-009/>, 2005.
- [14] W.H. Payne, J.R. Rabung, and T.P. Bogoy, "Coding the Lehmer Pseudo-Random Number Generator," *Comm. ACM*, 1969.
- [15] R. Raghavan and J. Hayes, "On Randomly Interleaved Memories," *Proc. 1990 Conf. Supercomputing*, 1990.
- [16] B.R. Rau, "Pseudo-Randomly Interleaved Memory," *Proc. 18th Int'l Symp. Computer Architecture*, 1991.
- [17] B.R. Rau, M. Schlansker, and D. Yen, "The Cydra 5 Stride-Insensitive Memory System," *Proc. Int'l Conf. Parallel Processing*, 1989.
- [18] A. Seznec, "A Case for Two-Way Skewed Associative Caches," *Proc. 20th Int'l Symp. Computer Architecture*, 1993.
- [19] A. Seznec, "A New Case for Skewed-Associativity," IRISA Technical Report #1114, 1997.
- [20] G.S. Sohi, "Logical Data Skewing Schemes for Interleaved Memories in Vector Processors," Univ. of Wisconsin-Madison Computer Science Technical Report #753, 1988.
- [21] Standard Performance Evaluation Corp., <http://www.spec.org>, 2005.
- [22] N. Topham, A. Gonzalez, and J. Gonzalez, "Eliminating Cache Conflict Misses through XOR-Based Placement Functions," *Proc. Int'l Conf. Supercomputing*, 1997.
- [23] N. Topham, A. Gonzalez, and J. Gonzalez, "The Design and Performance of a Conflict-Avoiding Cache," *Proc. Int'l Symp. Microarchitecture*, 1997.
- [24] P.-C. Wu, "Multiplicative, Congruential Random Number Generators with Multiplier $\pm 2^{k_1} \pm 2^{k_2}$ and Modulus $2^p - 1$," *ACM Trans. Math. Software*, 1997.
- [25] Q. Yang and L.W. Yang, "A Novel Cache Design for Vector Processing," *Proc. Int'l Symp. Computer Architecture*, 1992.
- [26] Z. Zhang, Z. Zhu, and X. Zhang, "A Permutation-Based Page Interleaving Scheme to Reduce Row-Buffer Conflicts and Exploit Data Locality," *Proc. Int'l Symp. Microarchitecture*, 2000.



Mazen Kharbutli received the BS degree in electrical and computer engineering from the Jordan University of Science and Technology in 2000, the MS degree in electrical engineering from the University of Maryland College Park in 2002, and is currently pursuing the PhD degree in computer engineering at North Carolina State University. He is an honored member of the Phi Kappa Phi honor society. His research interests include memory systems, computer architecture, and security and reliability in computer systems. He is a student member of the IEEE. More information can be found at <http://www4.ncsu.edu/mmkharbu>.



Yan Solihin received the BS degree in computer science from Institut Teknologi Bandung in 1995, the MASc degree in computer engineering from Nanyang Technological University in 1997, and the MS and PhD degrees in computer science from the University of Illinois at Urbana-Champaign in 1999 and 2002. He is currently an assistant professor in the Department of Electrical and Computer Engineering at North Carolina State University. From 1999 to 2000, he was on an internship with the Parallel Architecture and Performance Team at Los Alamos National Laboratory. He has published more than 25 papers in computer architecture and image processing which cover chip multiprocessor systems, processing-in-memory architectures, performance modeling, and architecture support for security and software reliability. He has released Scaltool, a software for pinpointing parallel program scalability bottlenecks, and Fodex, a forensic document examination toolset. He was a recipient of a 2004 US National Science Foundation Faculty Early Career Award and 1997 AT&T Leadership Award. He is a member of the IEEE. More information can be found at <http://www.cesr.ncsu.edu/solihin>.



Jaejin Lee received the BS degree in physics from Seoul National University in 1991, the MS degree in computer science from Stanford University in 1995, and the PhD degree in computer science from the University of Illinois at Urbana-Champaign in 1999. He is an associate professor in the School of Computer Science and Engineering at Seoul National University, Korea, where he has been a faculty member since September 2002. Before joining Seoul National University, he was an assistant professor in the Computer Science and Engineering Department at Michigan State University. His research interests include compilers, computer architectures, and embedded computer systems. He is a member of the IEEE and the ACM. More information can be found at <http://aces.snu.ac.kr/jlee>.